

PRM3

Théorie : Leçon 12

Java :

Introductions aux classes, aux objets et aux attributs

Les classes

Jusqu'à présent, les variables utilisées ne représentaient que des informations du même type. Les tableaux regroupent plusieurs informations mais, également, du même type.

Une **classe** (**class**) permet de **définir un type** qui regroupe **différents attributs** de n'importe quel type.

Exemple d'une **classe** **Personne** en Java, dans le fichier `Personne.java` :

```
public class Personne {
    public String nom;
    public int age;
}
```

Chaque personne aura deux attributs : son nom (un String) et son âge (un entier).

En Java, on écrira à priori une classe par fichier et ce fichier portera le même nom que la classe.

```
Programme.java
public class Programme {
    public static void main(String[] args) {
        Personne p1 = new Personne();
        p1.nom = "Jean";
        p1.age = 40;

        System.out.println(p1.nom);

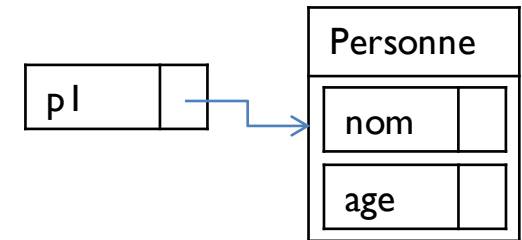
        Personne p2 = new Personne();
        p2.nom = "Paul";
        p2.age = 25;

        System.out.println(p2.nom + " " + p2.age);
    }
}
```

```
Personne.java
public class Personne {
    public String nom;
    public int age;
}
```

Variables

Objets



L'instruction `new Personne()` **construit** une **nouvelle instance** (objet) de la classe `Personne`.

La référence de cette instance est assignée à la **variable** `p1`, de type `Personne`.

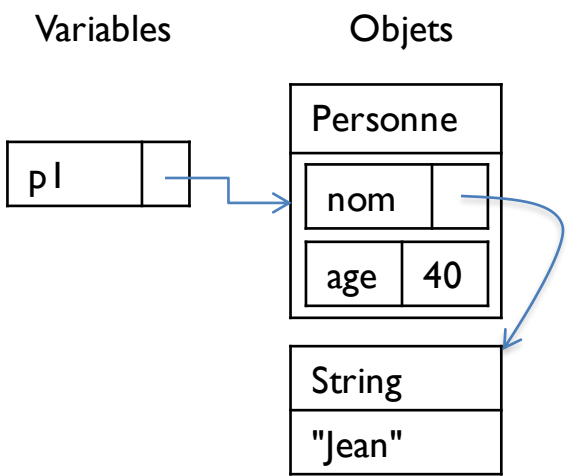
```
Programme.java
public class Programme {
    public static void main(String[] args) {
        Personne p1 = new Personne();
        p1.nom = "Jean";
        p1.age = 40;

        System.out.println(p1.nom);

        Personne p2 = new Personne();
        p2.nom = "Paul";
        p2.age = 25;

        System.out.println(p2.nom + " " + p2.age);
    }
}
```

```
Personne.java
public class Personne {
    public String nom;
    public int age;
}
```



On accède aux attributs à l'aide d'un point appliqué sur le référent de l'instance de la classe personne.

```
Programme.java
public class Programme {
    public static void main(String[] args) {
        Personne p1 = new Personne();
        p1.nom = "Jean";
        p1.age = 40;

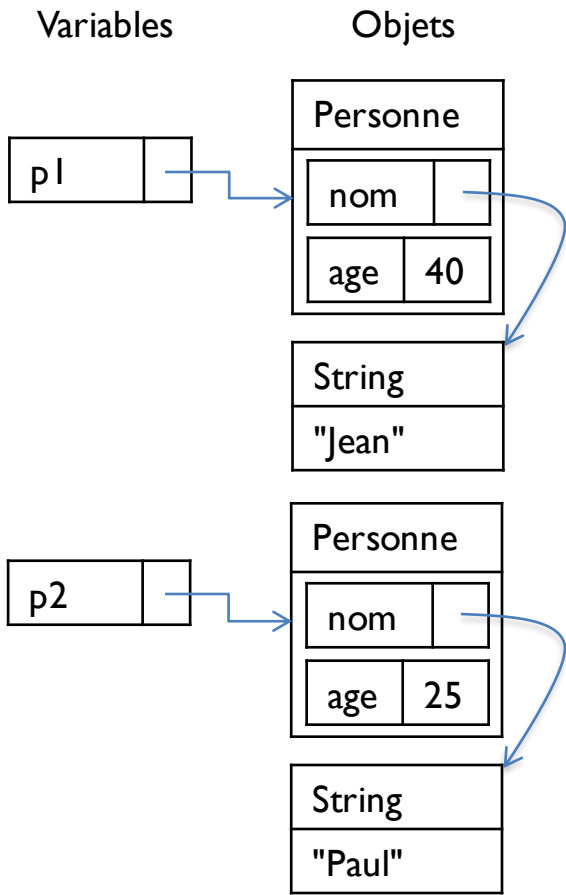
        System.out.println(p1.nom);

        Personne p2 = new Personne();
        p2.nom = "Paul";
        p2.age = 25;

        System.out.println(p2.nom + " " + p2.age);
    }
}
```

```
Personne.java
public class Personne {
    public String nom;
    public int age;
}
```

On peut construire **plusieurs instances** (objets) de la même classe.



```
Programme.java
public class Programme {
    public static void main(String[] args) {
        Personne p1 = new Personne();
        p1.nom = "Jean";
        p1.age = 40;

        Personne p2 = new Personne();
        p2.nom = "Paul";
        p2.age = 25;

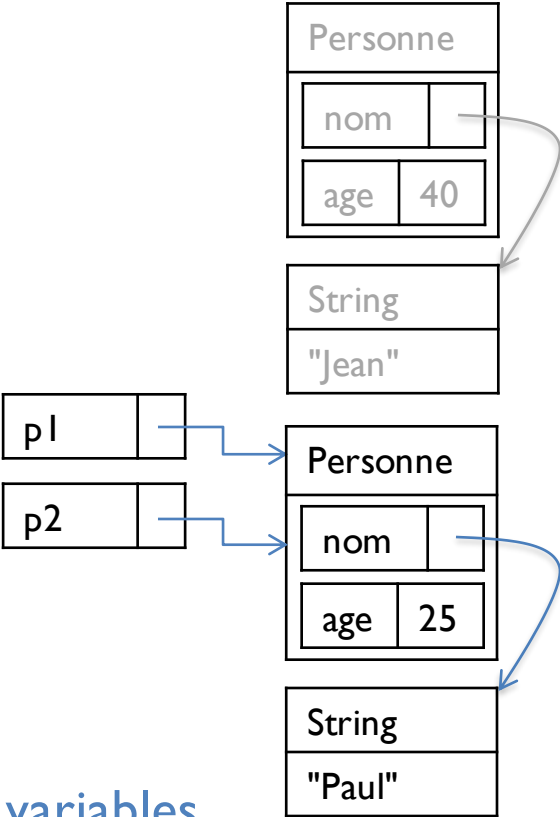
        p1 = p2;

        p1.nom = "Jacques";

        System.out.println(p2.nom);
    }
}
```

Variables

Objets



Un même objet peut être référencé par plusieurs variables.
Un objet qui n'est plus référencé n'est plus utilisable.

```

Programme.java
public class Programme {
    public static void main(String[] args) {
        Personne p1 = new Personne();
        p1.nom = "Jean";
        p1.age = 40;

        Personne p2 = new Personne();
        p2.nom = "Paul";
        p2.age = 25;

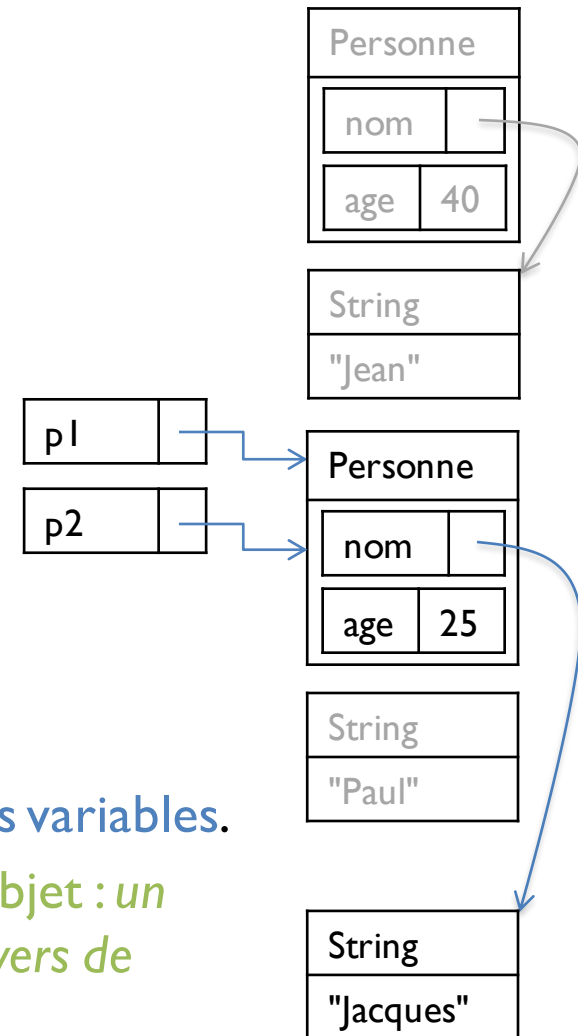
        p1 = p2;

        p1.nom = "Jacques";

        System.out.println(p2.nom);
    }
}
    
```

Variables

Objets



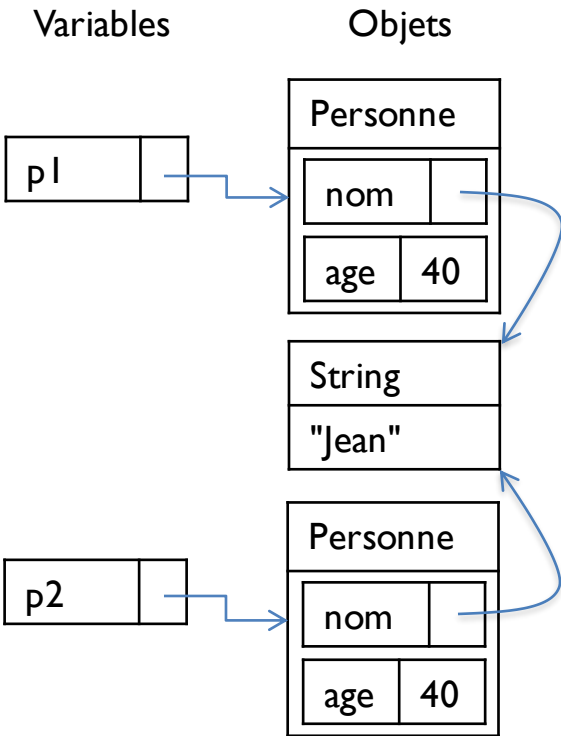
Un même objet peut être référencé par plusieurs variables.

Remarquez que `p1` et `p2` référencent le même objet : un changement "au travers de `p1`" se répercute "au travers de `p2`".

Programme.java

```
public class Programme {  
  
    public static void main(String[] args) {  
        Personne p1 = new Personne();  
        p1.nom = "Jean";  
        p1.age = 40;  
  
        Personne p2 = new Personne();  
        p2.nom = p1.nom;  
        p2.age = p1.age;  
  
        System.out.println(p1 == p2); //false  
        System.out.println(p1.nom == p2.nom); //true  
        System.out.println(p1.age == p2.age); //true  
    }  
}
```

L'adresse de deux objets différents est forcément différente... même si ces deux objets semblent identiques !



```
Programme.java
public class Programme {

    public static void affiche(Personne p){
        System.out.println(p.nom + " a " + p.age + " ans");
    }

    public static void anniversaire(Personne p){
        ++p.age;
    }

    public static void main(String[] args) {
        Personne p1 = new Personne();
        p1.nom = "Jean";
        p1.age = 40;

        affiche(p1);
        anniversaire(p1);
        affiche(p1);
    }
}
```

```
Personne.java
public class Personne {
    public String nom;
    public int age;
}
```

Le programme affiche :

Jean a 40 ans
Jean a 41 ans

A priori, les **objets** sont **muables** : leurs attributs peuvent **changer**.

Cela implique, comme pour les tableaux, que lorsque l'on passe la référence d'un objet muable à une **fonction**, celle-ci **peut modifier ses attributs**.

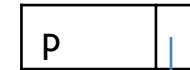
```

Programme.java
public class Programme {
    public static void main(String[] args) {
        Personne p = new Personne();
        p.nom = "Jean";
        p.frere.nom = "Luc";
    }
}
    
```

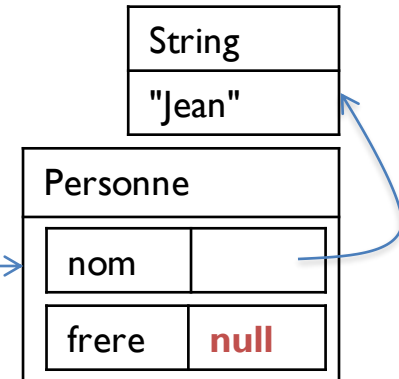
```

Personne.java
public class Personne {
    public String nom;
    public Personne frere;
}
    
```

Variables



Objets



Les attributs de type référence sont nuls (null) si on ne leur assigne rien. C'est à dire qu'aucun objet n'est référencé par l'attribut.

A l'exécution de la ligne encadrée, on essaye donc d'obtenir un attribut (nom) d'une référence nulle (p.frere). Une **NullPointerException** est alors déclenchée et le programme s'arrête.

Pour éviter cela, il aurait fallu assigner une instance de Personne à p.frere :
 p.frere = new Personne();